

Cognizant – upGrad Training Program

uGE_DotNet FSD (Python)

Weekly Submission Document

Week 3

Trainee Information

Name: Golla Amulya

Email: gollaamulya418@gmail.com

Batch Name: B1_Python

Course: uGE_DotNet FSD (Python)

Training Duration: 16th Feb 2026 – 26th May 2026

Session Timing: 9:30 AM – 6:30 PM

Faculty: Kunal

Week – 3

Tech Stack Covered: CSharp



1st CODE – Student Result Analyzer

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        // Store marks in List
        List<int> marks = new List<int> { 85, 70, 60, 75, 90 };

        // Call method with List
        double avg1 = CalculateAverage(marks);
        Console.WriteLine("Average (List): " + avg1);
        PrintGrade(avg1);

        // Call overloaded method with parameters
        double avg2 = CalculateAverage(85, 70, 60, 75, 90);
        Console.WriteLine("Average (Parameters): " + avg2);
        PrintGrade(avg2);
    }

    // Method 1: Using List
    static double CalculateAverage(List<int> marks)
    {
        int sum = 0;

        foreach (int mark in marks)
        {
            sum += mark;
        }

        return (double)sum / marks.Count;
    }

    // Method 2: Overloaded Method
    static double CalculateAverage(int m1, int m2, int m3, int m4, int m5)
    {
```

```

    int sum = m1 + m2 + m3 + m4 + m5;
    return (double)sum / 5;
}

// Grade method
static void PrintGrade(double avg)
{
    if (avg > 80)
        Console.WriteLine("Grade: A");
    else if (avg > 60)
        Console.WriteLine("Grade: B");
    else if (avg > 40)
        Console.WriteLine("Grade: C");
    else
        Console.WriteLine("Grade: Fail");
}
}

```

2nd CODE – Custom Calculator with Operation Method

```

using System;

class Program
{
    static void Main()
    {
        double result1 = Calculate(10, 5, "add");
        Console.WriteLine("Add Result: " + result1);

        double result2 = Calculate(10, 5, "subtract");
        Console.WriteLine("Subtract Result: " + result2);

        double result3 = Calculate(10, 5, "multiply");
        Console.WriteLine("Multiply Result: " + result3);

        double result4 = Calculate(10, 5, "divide");
        Console.WriteLine("Divide Result: " + result4);
    }

    static double Calculate(double a, double b, string operation)
    {
        if (operation == "divide" && b == 0)

```

```

{
    Console.WriteLine("Error: Cannot divide by zero");
    return 0;
}

return operation switch
{
    "add" => a + b,
    "subtract" => a - b,
    "multiply" => a * b,
    "divide" => a / b,
    _ => throw new ArgumentException("Invalid operation")
};
}
}

```

3rd CODE – Recursive Power Calculator

```

using System;

class Program
{
    static void Main()
    {
        int result = Power(2, 3);
        Console.WriteLine("Result: " + result);
    }

    static int Power(int baseNum, int exponent)
    {
        // Base condition
        if (exponent == 0)
            return 1;

        // Recursive call
        return baseNum * Power(baseNum, exponent - 1);
    }
}

```

4th CODE - Dictionary-Based Product Inventory

```

using System;

```

```
using System.Collections.Generic;
```

```
class Program
```

```
{  
    static Dictionary<string, double> products = new Dictionary<string, double>();
```

```
    static void Main()
```

```
    {  
        AddProduct("Laptop", 55000);  
        AddProduct("Mouse", 500);  
        AddProduct("Keyboard", 1200);
```

```
        DisplayAllProducts();
```

```
        Console.WriteLine("Price of Mouse: " + FindProductPrice("Mouse"));
```

```
        RemoveProduct("Mouse");
```

```
        DisplayAllProducts();
```

```
    }
```

```
// Add product
```

```
static void AddProduct(string name, double price)
```

```
{  
    products[name] = price;  
    Console.WriteLine(name + " added successfully.");  
}
```

```
// Remove product
```

```
static void RemoveProduct(string name)
```

```
{  
    if (products.Remove(name))  
        Console.WriteLine(name + " removed successfully.");  
    else  
        Console.WriteLine("Product not found.");  
}
```

```
// Display all products
```

```
static void DisplayAllProducts()
```

```
{  
    Console.WriteLine("\nProduct List:");
```

```

foreach (var item in products)
{
    Console.WriteLine(item.Key + " : " + item.Value);
}

// Find product price
static double FindProductPrice(string name)
{
    if (products.ContainsKey(name))
        return products[name];

    Console.WriteLine("Product not found.");
    return 0;
}
}

```

5th CODE - Student Management using Class and List

```

using System;
using System.Collections.Generic;

```

```

class Student
{
    public int Id;
    public string Name;
    public int Marks;

    public string CalculateGrade()
    {
        if (Marks >= 85)
            return "A";
        else if (Marks >= 70)
            return "B";
        else if (Marks >= 50)
            return "C";
        else
            return "Fail";
    }
}

```

```

class Program

```

```

{
    static void Main()
    {
        List<Student> students = new List<Student>();

        students.Add(new Student { Id = 1, Name = "Ravi", Marks = 90 });
        students.Add(new Student { Id = 2, Name = "Anita", Marks = 75 });
        students.Add(new Student { Id = 3, Name = "Kiran", Marks = 55 });
        students.Add(new Student { Id = 4, Name = "Meena", Marks = 40 });

        Console.WriteLine("Student Details:");

        foreach (Student s in students)
        {
            Console.WriteLine(
                "ID: " + s.Id +
                ", Name: " + s.Name +
                ", Marks: " + s.Marks +
                ", Grade: " + s.CalculateGrade()
            );
        }
    }
}

```

6th CODE - Product Class with Constructor Overloading

```

using System;

class Product
{
    public string Name;
    public double Price;
    public int Quantity;

    // Default Constructor
    public Product()
    {
        Name = "Unknown";
        Price = 0;
        Quantity = 0;
    }
}

```

```

// Parameterized Constructor
public Product(string name, double price, int quantity)
{
    Name = name;
    Price = price;
    Quantity = quantity;
}

// Method to calculate total price
public double GetTotalPrice()
{
    return Price * Quantity;
}
}

class Program
{
    static void Main()
    {
        // Using default constructor
        Product p1 = new Product();
        Console.WriteLine("Product 1: " + p1.Name + ", Total Price: " + p1.GetTotalPrice());

        // Using parameterized constructor
        Product p2 = new Product("Laptop", 50000, 2);
        Console.WriteLine("Product 2: " + p2.Name + ", Total Price: " + p2.GetTotalPrice());
    }
}

```

7th CODE - Encapsulation with Validation

```

using System;

class Employee
{
    // Private fields (Data Hiding)
    private double salary;
    private int age;

    // Property for Salary
    public double Salary
    {

```

```

    get { return salary; }
    set
    {
        if (value > 0)
            salary = value;
        else
            Console.WriteLine("Invalid Salary. Salary must be greater than 0.");
    }
}

// Property for Age
public int Age
{
    get { return age; }
    set
    {
        if (value >= 18 && value <= 60)
            age = value;
        else
            Console.WriteLine("Invalid Age. Age must be between 18 and 60.");
    }
}
}

class Program
{
    static void Main()
    {
        Employee emp = new Employee();

        emp.Salary = 50000;
        emp.Age = 25;

        Console.WriteLine("Salary: " + emp.Salary);
        Console.WriteLine("Age: " + emp.Age);

        // Invalid values
        emp.Salary = -1000;
        emp.Age = 70;
    }
}

```

using System;

using System.Collections.Generic;

// Parent abstract class

abstract class Shape

```
{  
    public abstract double CalculateArea();  
}
```

// Circle class

class Circle : Shape

```
{  
    public double Radius;  
  
    public Circle(double radius)  
    {  
        Radius = radius;  
    }  
  
    public override double CalculateArea()  
    {  
        return Math.PI * Radius * Radius;  
    }  
}
```

// Rectangle class

class Rectangle : Shape

```
{  
    public double Length;  
    public double Width;  
  
    public Rectangle(double length, double width)  
    {  
        Length = length;  
        Width = width;  
    }  
  
    public override double CalculateArea()  
    {  
        return Length * Width;  
    }  
}
```

```

    }
}

// Triangle class
class Triangle : Shape
{
    public double Base;
    public double Height;

    public Triangle(double b, double h)
    {
        Base = b;
        Height = h;
    }

    public override double CalculateArea()
    {
        return 0.5 * Base * Height;
    }
}

class Program
{
    static void Main()
    {
        List<Shape> shapes = new List<Shape>();

        shapes.Add(new Circle(5));
        shapes.Add(new Rectangle(4, 6));
        shapes.Add(new Triangle(3, 8));

        foreach (Shape s in shapes)
        {
            Console.WriteLine("Area: " + s.CalculateArea());
        }
    }
}

```

```

using System;
using System.Collections.Generic;

```

```
// Parent class
class Account
{
    public double Balance;

    public Account(double balance)
    {
        Balance = balance;
    }

    public virtual double CalculateInterest()
    {
        return 0;
    }
}

// Child class - Savings Account
class SavingsAccount : Account
{
    public SavingsAccount(double balance) : base(balance) { }

    public override double CalculateInterest()
    {
        return Balance * 0.04; // 4% interest
    }
}

// Child class - Current Account
class CurrentAccount : Account
{
    public CurrentAccount(double balance) : base(balance) { }

    public override double CalculateInterest()
    {
        return Balance * 0.02; // 2% interest
    }
}

class Program
{
    static void Main()
```

```

{
    List<Account> accounts = new List<Account>();

    accounts.Add(new SavingsAccount(10000));
    accounts.Add(new CurrentAccount(10000));

    foreach (Account acc in accounts)
    {
        Console.WriteLine("Interest: " + acc.CalculateInterest());
    }
}

```

10th CODE - Vehicle Service System using Polymorphism + Encapsulation

```

using System;
using System.Collections.Generic;

// Parent class
class Vehicle
{
    // Encapsulation
    private string vehicleName;

    public string VehicleName
    {
        get { return vehicleName; }
        set { vehicleName = value; }
    }

    public void StartEngine()
    {
        Console.WriteLine(VehicleName + " engine started.");
    }

    public virtual double ServiceCost()
    {
        return 0;
    }
}

// Car class

```

```
class Car : Vehicle
{
    public override double ServiceCost()
    {
        return 5000;
    }
}
```

// Bike class

```
class Bike : Vehicle
{
    public override double ServiceCost()
    {
        return 1500;
    }
}
```

// Truck class

```
class Truck : Vehicle
{
    public override double ServiceCost()
    {
        return 8000;
    }
}
```

class Program

```
{
    static void Main()
    {
        List<Vehicle> vehicles = new List<Vehicle>();

        vehicles.Add(new Car { VehicleName = "Car" });
        vehicles.Add(new Bike { VehicleName = "Bike" });
        vehicles.Add(new Truck { VehicleName = "Truck" });

        double totalCost = 0;

        foreach (Vehicle v in vehicles)
        {
            v.StartEngine();
            double cost = v.ServiceCost();
        }
    }
}
```

```

        Console.WriteLine("Service Cost: " + cost);

        totalCost += cost;
    }

    Console.WriteLine("Total Service Cost: " + totalCost);
}
}

```

Week – 3 | Date: 03/03/2026 and 04/03/2026 | Day -2,3

1st CODE - Even-Odd Transformation Pipeline

```

using System;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] numbers = { 3, 6, 9, 3, 12, 15, 6, 4, 7 };

        var result = numbers
            .Distinct()           // Remove duplicates
            .Where(n => n % 3 == 0) // Keep numbers divisible by 3
            .Select(n => n * n)    // Square them
            .OrderBy(n => n);      // Sort ascending

        Console.WriteLine(string.Join(", ", result));
    }
}

```

2nd CODE - Custom Stack with History Tracking

```

using System;
using System.Collections.Generic;

// Custom Exception
class EmptyStackException : Exception
{

```

```
    public EmptyStackException(string message) : base(message) {}  
}
```

```
class CustomStack
```

```
{  
    private Stack<int> stack = new Stack<int>();  
    private Stack<string> history = new Stack<string>();
```

```
    // Push
```

```
    public void Push(int value)
```

```
{  
    stack.Push(value);  
    history.Push("Push:" + value);  
}
```

```
    // Pop
```

```
    public int Pop()
```

```
{  
    if (stack.Count == 0)  
        throw new EmptyStackException("Stack is empty!");  
  
    int value = stack.Pop();  
    history.Push("Pop:" + value);  
    return value;  
}
```

```
    // Peek
```

```
    public int Peek()
```

```
{  
    if (stack.Count == 0)  
        throw new EmptyStackException("Stack is empty!");  
  
    return stack.Peek();  
}
```

```
    // Undo last operation
```

```
    public void UndoLastOperation()
```

```
{  
    if (history.Count == 0)  
    {  
        Console.WriteLine("No operations to undo.");  
        return;  
    }  
}
```

```

    }

    string last = history.Pop();
    string[] parts = last.Split(':');
    string operation = parts[0];
    int value = int.Parse(parts[1]);

    if (operation == "Push")
    {
        stack.Pop();
        Console.WriteLine("Undo Push of " + value);
    }
    else if (operation == "Pop")
    {
        stack.Push(value);
        Console.WriteLine("Undo Pop of " + value);
    }
}

}

class Program
{
    static void Main()
    {
        CustomStack cs = new CustomStack();

        cs.Push(10);
        cs.Push(20);
        cs.Push(30);

        Console.WriteLine("Peek: " + cs.Peek());

        Console.WriteLine("Pop: " + cs.Pop());

        cs.UndoLastOperation();

        Console.WriteLine("Peek after undo: " + cs.Peek());
    }
}

```

```

using System;
using System.Collections.Generic;

class TaskScheduler
{
    static void Main()
    {
        Queue<string> high = new Queue<string>();
        Queue<string> medium = new Queue<string>();
        Queue<string> low = new Queue<string>();

        // Adding tasks
        AddTask("Email Client", 2, high, medium, low);
        AddTask("Fix Production Bug", 1, high, medium, low);
        AddTask("Write Documentation", 3, high, medium, low);
        AddTask("Deploy Update", 1, high, medium, low);

        // Processing tasks
        ProcessTasks(high, medium, low);
    }

    static void AddTask(string task, int priority,
        Queue<string> high, Queue<string> medium, Queue<string> low)
    {
        if (priority == 1)
            high.Enqueue(task);
        else if (priority == 2)
            medium.Enqueue(task);
        else
            low.Enqueue(task);
    }

    static void ProcessTasks(Queue<string> high,
        Queue<string> medium, Queue<string> low)
    {
        while (high.Count > 0)
            Console.WriteLine("Processing: " + high.Dequeue());

        while (medium.Count > 0)
            Console.WriteLine("Processing: " + medium.Dequeue());

        while (low.Count > 0)

```

```
        Console.WriteLine("Processing: " + low.Dequeue());
    }
}
```

4th CODE - Improved HashMap with Collision Detection

```
using System;
using System.Collections.Generic;

class HashNode
{
    public string Key;
    public int Value;

    public HashNode(string key, int value)
    {
        Key = key;
        Value = value;
    }
}

class SimpleHashMap
{
    private List<HashNode>[] buckets;
    private int size;

    public SimpleHashMap(int size)
    {
        this.size = size;
        buckets = new List<HashNode>[size];
    }

    private int GetIndex(string key)
    {
        int hash = key.GetHashCode();
        return Math.Abs(hash % size);
    }

    public void Add(string key, int value)
    {
        int index = GetIndex(key);
```

```

        if (buckets[index] == null)
            buckets[index] = new List<HashNode>();
        else
            Console.WriteLine("⚠ Collision detected at index " + index);

        buckets[index].Add(new HashNode(key, value));
    }

    public int? Get(string key)
    {
        int index = GetIndex(key);

        if (buckets[index] != null)
        {
            foreach (var node in buckets[index])
            {
                if (node.Key == key)
                    return node.Value;
            }
        }

        return null;
    }

    public void Remove(string key)
    {
        int index = GetIndex(key);

        if (buckets[index] != null)
        {
            buckets[index].RemoveAll(n => n.Key == key);
        }
    }
}

class Program
{
    static void Main()
    {
        SimpleHashMap map = new SimpleHashMap(5);

        map.Add("A", 10);
    }
}

```

```

map.Add("B", 20);
map.Add("C", 30);

Console.WriteLine(map.Get("B"));

map.Remove("B");

Console.WriteLine(map.Get("B"));
}
}

```

5th CODE - Binary Tree – Count & Height

```

using System;

class Node
{
    public int Data;
    public Node Left;
    public Node Right;

    public Node(int data)
    {
        Data = data;
        Left = null;
        Right = null;
    }
}

class BinaryTree
{
    public Node Root;

    // Count Nodes
    public int CountNodes(Node node)
    {
        if (node == null)
            return 0;

        return 1 + CountNodes(node.Left) + CountNodes(node.Right);
    }
}

```

```

// Get Height
public int GetHeight(Node node)
{
    if (node == null)
        return 0;

    int leftHeight = GetHeight(node.Left);
    int rightHeight = GetHeight(node.Right);

    return Math.Max(leftHeight, rightHeight) + 1;
}

// Find Minimum
public int FindMin(Node node)
{
    if (node.Left == null)
        return node.Data;

    return FindMin(node.Left);
}

// Find Maximum
public int FindMax(Node node)
{
    if (node.Right == null)
        return node.Data;

    return FindMax(node.Right);
}
}

```

```

class Program
{
    static void Main()
    {
        BinaryTree tree = new BinaryTree();

        tree.Root = new Node(10);
        tree.Root.Left = new Node(5);
        tree.Root.Right = new Node(20);
        tree.Root.Left.Left = new Node(3);
        tree.Root.Left.Right = new Node(7);
    }
}

```

```

tree.Root.Right.Right = new Node(30);

Console.WriteLine("Total Nodes: " + tree.CountNodes(tree.Root));
Console.WriteLine("Height: " + tree.GetHeight(tree.Root));
Console.WriteLine("Min Value: " + tree.FindMin(tree.Root));
Console.WriteLine("Max Value: " + tree.FindMax(tree.Root));
}
}

```

6th CODE - Student File Processor with Error Logging

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

// Custom Exception
class InvalidMarksException : Exception
{
    public InvalidMarksException(string message) : base(message)
    {
    }
}

// Student Class
class Student
{
    public string Name { get; set; }
    public int Marks { get; set; }
}

class Program
{
    static void Main()
    {
        List<Student> students = new List<Student>();

        string[] lines = File.ReadAllLines("students.txt");

        foreach (string line in lines)
        {
            try

```

```

{
    string[] parts = line.Split(',');

    string name = parts[0];
    string markText = parts[1];

    int marks;

    if (!int.TryParse(markText, out marks))
    {
        throw new InvalidMarksException($"Invalid marks for {name}: {markText}");
    }

    students.Add(new Student { Name = name, Marks = marks });
}
catch (InvalidMarksException ex)
{
    File.AppendAllText("errors.txt", ex.Message + Environment.NewLine);
}
}

// Calculate Average using LINQ
if (students.Count > 0)
{
    double average = students.Average(s => s.Marks);
    Console.WriteLine("Average Marks: " + average);
}
else
{
    Console.WriteLine("No valid student data.");
}
}
}

```

7th CODE - Nullable Order Discount System

using System;

class Program

```

{
    static void Main()
    {

```

```

double orderAmount = 6000;

double? discountPercentage = null; // Nullable discount

try
{
    double discount = discountPercentage ?? 10; // default 10%

    if (discount > 40)
    {
        throw new Exception("Discount cannot be more than 40%");
    }

    double finalPrice = orderAmount - (orderAmount * discount / 100);

    // Pattern matching
    switch (orderAmount)
    {
        case > 5000:
            finalPrice -= orderAmount * 0.05;
            Console.WriteLine("Extra 5% discount applied");
            break;

        case <= 5000:
            Console.WriteLine("No extra discount");
            break;
    }

    Console.WriteLine("Final Price: " + finalPrice);
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
}

```

8th CODE - Employee Analytics using LINQ

```

using System;
using System.Collections.Generic;
using System.Linq;

```

```

class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
    public double Salary { get; set; }
}

class Program
{
    static void Main()
    {
        List<Employee> employees = new List<Employee>()
        {
            new Employee { Name="Rahul", Department="IT", Salary=50000 },
            new Employee { Name="Kunal", Department="HR", Salary=40000 },
            new Employee { Name="Amit", Department="IT", Salary=70000 },
            new Employee { Name="Neha", Department="HR", Salary=45000 },
            new Employee { Name="Priya", Department="Finance", Salary=60000 }
        };

        // Average Salary
        double avgSalary = employees.Average(e => e.Salary);

        // Employees with salary > average
        var highSalaryEmployees = employees
            .Where(e => e.Salary > avgSalary);

        Console.WriteLine("Employees with salary above average:");

        foreach (var emp in highSalaryEmployees)
        {
            Console.WriteLine(emp.Name + " - " + emp.Salary);
        }

        // Group by department and find highest salary employee
        Dictionary<string, Employee> result =
            employees
                .GroupBy(e => e.Department)
                .ToDictionary(
                    g => g.Key,
                    g => g.OrderByDescending(e => e.Salary).First()
                )
    }
}

```

```

    );

    Console.WriteLine("\nHighest salary employee per department:");

    foreach (var item in result)
    {
        Console.WriteLine(item.Key + " : " + item.Value.Name + " - " + item.Value.Salary);
    }
}
}

```

9th CODE - Tuple-Based Result Evaluation System

```

using System;
using System.Collections.Generic;
using System.IO;

class Program
{
    static void Main()
    {
        List<(string name, int marks)> students = new List<(string, int)>()
        {
            ("Rahul",95),
            ("Kunal",80),
            ("Amit",65),
            ("Neha",35)
        };

        string filePath = "results.txt";

        foreach (var student in students)
        {
            // Tuple deconstruction
            (string grade, bool isPass) = Evaluate(student.marks);

            string line = $"{student.name},{student.marks},{grade},{isPass}";
            File.AppendAllText(filePath, line + Environment.NewLine);
        }

        Console.WriteLine("Passed Students:");
    }
}

```

```

string[] lines = File.ReadAllLines(filePath);

foreach (var line in lines)
{
    string[] parts = line.Split(',');

    string name = parts[0];
    string grade = parts[2];
    bool isPass = bool.Parse(parts[3]);

    if (isPass)
    {
        Console.WriteLine($"{name} - Grade: {grade}");
    }
}

// Tuple returning method
static (string grade, bool isPass) Evaluate(int marks)
{
    if (marks >= 90)
        return ("A+", true);
    else if (marks >= 75)
        return ("A", true);
    else if (marks >= 60)
        return ("B", true);
    else if (marks >= 40)
        return ("C", true);
    else
        return ("Fail", false);
}
}

```

10th CODE - Mini File-Based CRUD Student System

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

```

```

class Student
{

```

```
public int Id { get; set; }  
public string Name { get; set; }  
public int Marks { get; set; }  
}
```

```
class Program
```

```
{  
    static string filePath = "students.txt";  
  
    static void Main()  
    {  
        while (true)  
        {  
            Console.WriteLine("\n1. Add Student");  
            Console.WriteLine("2. View All Students");  
            Console.WriteLine("3. Search Student by Id");  
            Console.WriteLine("4. Delete Student");  
            Console.WriteLine("5. Exit");  
  
            Console.WriteLine("Choose option: ");  
            int choice = Convert.ToInt32(Console.ReadLine());  
  
            try  
            {  
                switch (choice)  
                {  
                    case 1:  
                        AddStudent();  
                        break;  
  
                    case 2:  
                        ViewStudents();  
                        break;  
  
                    case 3:  
                        SearchStudent();  
                        break;  
  
                    case 4:  
                        DeleteStudent();  
                        break;  
                }  
            }  
        }  
    }  
}
```

```

        case 5:
            return;

        default:
            Console.WriteLine("Invalid option");
            break;
    }
}
catch (Exception ex)
{
    Console.WriteLine("Error: " + ex.Message);
}
}

static void AddStudent()
{
    Console.Write("Enter Id: ");
    int id = Convert.ToInt32(Console.ReadLine());

    Console.Write("Enter Name: ");
    string name = Console.ReadLine();

    Console.Write("Enter Marks: ");
    int marks = Convert.ToInt32(Console.ReadLine());

    string line = $"{id},{name},{marks}";

    File.AppendAllText(filePath, line + Environment.NewLine);

    Console.WriteLine("Student added successfully.");
}

static void ViewStudents()
{
    if (!File.Exists(filePath))
    {
        Console.WriteLine("No data found.");
        return;
    }

    string[] lines = File.ReadAllLines(filePath);

```

```

var students = lines.Select(line =>
{
    var parts = line.Split(',');
    return new Student
    {
        Id = int.Parse(parts[0]),
        Name = parts[1],
        Marks = int.Parse(parts[2])
    };
});

foreach (var s in students)
{
    Console.WriteLine($"{s.Id} - {s.Name} - {s.Marks}");
}
}

static void SearchStudent()
{
    Console.Write("Enter Id to search: ");
    int id = Convert.ToInt32(Console.ReadLine());

    string[] lines = File.ReadAllLines(filePath);

    var student = lines
        .Select(line =>
        {
            var parts = line.Split(',');
            return new Student
            {
                Id = int.Parse(parts[0]),
                Name = parts[1],
                Marks = int.Parse(parts[2])
            };
        })
        .FirstOrDefault(s => s.Id == id);

    if (student != null)
        Console.WriteLine($"{student.Id} - {student.Name} - {student.Marks}");
    else
        Console.WriteLine("Student not found.");
}

```

```

}

static void DeleteStudent()
{
    Console.WriteLine("Enter Id to delete: ");
    int id = Convert.ToInt32(Console.ReadLine());

    var lines = File.ReadAllLines(filePath).ToList();

    var updated = lines.Where(line => !line.StartsWith(id + ",")).ToList();

    File.WriteAllLines(filePath, updated);

    Console.WriteLine("Student deleted if existed.");
}
}

```

Week – 3 | Date: 05/03/2026 and 06/03/2026 | Day -4,5

1st CODE - Thread-Safe Order Processing System

```

using System;
using System.Collections.Generic;
using System.Threading;

class Order
{
    public int OrderId { get; set; }
    public string Product { get; set; }
}

class OrderService
{
    private List<Order> orders = new List<Order>();
    private readonly object lockObject = new object();

    public void PlaceOrder(int orderId, string product)
    {
        lock (lockObject) // Prevent race condition
        {
            // Check duplicate OrderId

```

```

        if (orders.Exists(o => o.OrderId == orderId))
        {
            Console.WriteLine($"Order {orderId} already exists.");
            return;
        }

        orders.Add(new Order { OrderId = orderId, Product = product });

        Console.WriteLine(
            $"Order {orderId} placed by Thread {Thread.CurrentThread.ManagedThreadId}"
        );
    }
}

public void ShowOrders()
{
    foreach (var order in orders)
    {
        Console.WriteLine($"{{order.OrderId}} - {{order.Product}}");
    }
}
}

class Program
{
    static void Main()
    {
        OrderService service = new OrderService();

        Thread t1 = new Thread(() => service.PlaceOrder(1, "Laptop"));
        Thread t2 = new Thread(() => service.PlaceOrder(2, "Mobile"));
        Thread t3 = new Thread(() => service.PlaceOrder(1, "Tablet")); // duplicate

        t1.Start();
        t2.Start();
        t3.Start();

        t1.Join();
        t2.Join();
        t3.Join();

        Console.WriteLine("\nAll Orders:");
    }
}

```

```
        service.ShowOrders();
    }
}
```

2nd CODE - Parallel File Processor with Timing

```
using System;
using System.IO;
using System.Linq;
using System.Diagnostics;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        string[] files =
        {
            "file1.txt",
            "file2.txt",
            "file3.txt",
            "file4.txt",
            "file5.txt"
        };

        // Sequential Processing
        Stopwatch sw1 = Stopwatch.StartNew();
        int sequentialCount = 0;

        foreach (var file in files)
        {
            string text = File.ReadAllText(file);
            int words = text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;

            sequentialCount += words;

            Debug.WriteLine($"Sequential processed: {file}");
        }

        sw1.Stop();

        Console.WriteLine("Sequential Word Count: " + sequentialCount);
    }
}
```

```

Console.WriteLine("Sequential Time: " + sw1.ElapsedMilliseconds + " ms");

Console.WriteLine();

// Parallel Processing
Stopwatch sw2 = Stopwatch.StartNew();
int parallelCount = 0;
object lockObj = new object();

Parallel.ForEach(files, file =>
{
    string text = File.ReadAllText(file);
    int words = text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;

    lock (lockObj)
    {
        parallelCount += words;
    }

    Debug.WriteLine($"Parallel processed: {file}");
});

sw2.Stop();

Console.WriteLine("Parallel Word Count: " + parallelCount);
Console.WriteLine("Parallel Time: " + sw2.ElapsedMilliseconds + " ms");
}
}

```

3rd CODE - Async Data Aggregator

```

using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        try
        {
            Console.WriteLine("Fetching data...");

```

```

var usersTask = FetchUsersAsync();
var ordersTask = FetchOrdersAsync();
var paymentsTask = FetchPaymentsAsync();

// Run all tasks concurrently
var results = await Task.WhenAll(usersTask, ordersTask, paymentsTask);

Console.WriteLine("\nCombined Results:");
foreach (var result in results)
{
    Console.WriteLine(result);
}
}
catch (Exception ex)
{
    Console.WriteLine("Error occurred while fetching data:");
    Console.WriteLine(ex.Message);
    Console.WriteLine("StackTrace: " + ex.StackTrace);
}
}

static async Task<string> FetchUsersAsync()
{
    await Task.Delay(2000); // simulate API delay
    return "Users data loaded";
}

static async Task<string> FetchOrdersAsync()
{
    await Task.Delay(3000);
    return "Orders data loaded";
}

static async Task<string> FetchPaymentsAsync()
{
    await Task.Delay(2500);

    // Simulate failure
    throw new Exception("Payment service failed");

    // return "Payments data loaded";
}

```

```
}
```

4th CODE - Runtime Failure Analyzer

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        Controller controller = new Controller();  
  
        try  
        {  
            controller.GetData();  
        }  
        catch (Exception ex)  
        {  
            Console.WriteLine("Unhandled Exception:");  
            Console.WriteLine(ex.Message);  
            Console.WriteLine(ex.StackTrace);  
        }  
    }  
}
```

```
class Controller
```

```
{  
    private Service service = new Service();  
  
    public void GetData()  
    {  
        Console.WriteLine("Entering Controller.GetData()");  
  
        try  
        {  
            service.ProcessData();  
        }  
        catch (Exception ex)  
        {  
            Console.WriteLine("Controller caught exception:");  
            Console.WriteLine(ex.Message);  
            throw;  
        }  
    }  
}
```

```

    }

    Console.WriteLine("Exiting Controller.GetData()");
}
}

class Service
{
    private Repository repo = new Repository();

    public void ProcessData()
    {
        Console.WriteLine("Entering Service.ProcessData()");

        try
        {
            repo.GetDataFromDatabase();
        }
        catch (Exception ex)
        {
            Console.WriteLine("Service caught exception:");
            Console.WriteLine(ex.Message);
            throw;
        }

        Console.WriteLine("Exiting Service.ProcessData()");
    }
}

class Repository
{
    public void GetDataFromDatabase()
    {
        Console.WriteLine("Entering Repository.GetDataFromDatabase()");

        // Simulated failure
        throw new Exception("Database connection failed!");

        // This line will never run
        // Console.WriteLine("Exiting Repository.GetDataFromDatabase()");
    }
}

```

```
using System;
using System.Data.SQLite;
using System.Security.Cryptography;
using System.Text;

class Program
{
    static string connectionString = "Data Source=users.db";

    static void Main()
    {
        CreateTable();

        Console.WriteLine("1. Register");
        Console.WriteLine("2. Login");
        Console.Write("Choose option: ");

        int choice = Convert.ToInt32(Console.ReadLine());

        if (choice == 1)
            Register();
        else if (choice == 2)
            Login();
    }

    static void CreateTable()
    {
        using var connection = new SQLiteConnection(connectionString);
        connection.Open();

        string sql =
            "CREATE TABLE IF NOT EXISTS Users (" +
            "Username TEXT PRIMARY KEY, " +
            "PasswordHash TEXT)";

        SQLiteCommand cmd = new SQLiteCommand(sql, connection);
        cmd.ExecuteNonQuery();
    }

    static void Register()
```

```

{
    Console.Write("Enter username: ");
    string username = Console.ReadLine();

    Console.Write("Enter password: ");
    string password = Console.ReadLine();

    if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
    {
        Console.WriteLine("Invalid input.");
        return;
    }

    string hash = HashPassword(password);

    try
    {
        using var connection = new SQLiteConnection(connectionString);
        connection.Open();

        string sql = "INSERT INTO Users (Username, PasswordHash) VALUES (@user, @pass)";

        SQLiteCommand cmd = new SQLiteCommand(sql, connection);
        cmd.Parameters.AddWithValue("@user", username);
        cmd.Parameters.AddWithValue("@pass", hash);

        cmd.ExecuteNonQuery();

        Console.WriteLine("User registered successfully.");
    }
    catch (SQLiteException ex)
    {
        Console.WriteLine("Database error: " + ex.Message);
    }
}

static void Login()
{
    Console.Write("Enter username: ");
    string username = Console.ReadLine();

    Console.Write("Enter password: ");

```

```

string password = Console.ReadLine();

string hash = HashPassword(password);

try
{
    using var connection = new SQLiteConnection(connectionString);
    connection.Open();

    string sql = "SELECT PasswordHash FROM Users WHERE Username=@user";

    SQLiteCommand cmd = new SQLiteCommand(sql, connection);
    cmd.Parameters.AddWithValue("@user", username);

    var result = cmd.ExecuteScalar();

    if (result != null && result.ToString() == hash)
        Console.WriteLine("Login successful");
    else
        Console.WriteLine("Invalid username or password");
}
catch (SQLiteException ex)
{
    Console.WriteLine("SQL Error: " + ex.Message);
}
}

static string HashPassword(string password)
{
    using SHA256 sha = SHA256.Create();

    byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));

    StringBuilder builder = new StringBuilder();

    foreach (var b in bytes)
        builder.Append(b.ToString("x2"));

    return builder.ToString();
}
}

```

```
using System;
using System.Data.SQLite;

class Program
{
    static void Main()
    {
        string connectionString = "Data Source=bank.db;Version=3;";

        // Step 1: Create Accounts table if it doesn't exist
        using (var conn = new SQLiteConnection(connectionString))
        {
            conn.Open();

            string createTable = @"
                CREATE TABLE IF NOT EXISTS Accounts (
                    AccountId INTEGER PRIMARY KEY,
                    Name TEXT NOT NULL,
                    Balance REAL NOT NULL
                );
            ";

            using (var cmd = new SQLiteCommand(createTable, conn))
            {
                cmd.ExecuteNonQuery();
            }

            // Step 2: Insert sample accounts (if not exists)
            string insertAccounts = @"
                INSERT OR IGNORE INTO Accounts (AccountId, Name, Balance) VALUES (1, 'Alice', 1000);
                INSERT OR IGNORE INTO Accounts (AccountId, Name, Balance) VALUES (2, 'Bob', 500);
            ";

            using (var cmd = new SQLiteCommand(insertAccounts, conn))
            {
                cmd.ExecuteNonQuery();
            }

            conn.Close();
        }
    }
}
```

```

// Step 3: Perform money transfer
TransferMoney(1, 2, 300, connectionString); // Transfer 300 from Alice to Bob
TransferMoney(1, 2, 1000, connectionString); // Attempt to transfer more than balance (should rollback)
}

static void TransferMoney(int fromAccountId, int toAccountId, double amount, string connectionString)
{
    using (var conn = new SQLiteConnection(connectionString))
    {
        conn.Open();
        using (var transaction = conn.BeginTransaction())
        {
            try
            {
                // 1. Check sender balance
                double senderBalance;
                using (var cmd = new SQLiteCommand("SELECT Balance FROM Accounts WHERE AccountId = @id",
conn))
                {
                    cmd.Parameters.AddWithValue("@id", fromAccountId);
                    senderBalance = Convert.ToDouble(cmd.ExecuteScalar());
                }

                if (senderBalance < amount)
                {
                    throw new Exception("Insufficient balance! Transaction aborted.");
                }

                // 2. Deduct from sender
                using (var cmd = new SQLiteCommand("UPDATE Accounts SET Balance = Balance - @amt WHERE
AccountId = @id", conn))
                {
                    cmd.Parameters.AddWithValue("@amt", amount);
                    cmd.Parameters.AddWithValue("@id", fromAccountId);
                    cmd.ExecuteNonQuery();
                }

                // 3. Add to receiver
                using (var cmd = new SQLiteCommand("UPDATE Accounts SET Balance = Balance + @amt WHERE
AccountId = @id", conn))
                {

```

```

        cmd.Parameters.AddWithValue("@amt", amount);
        cmd.Parameters.AddWithValue("@id", toAccountId);
        cmd.ExecuteNonQuery();
    }

    // ✅ Commit transaction
    transaction.Commit();
    Console.WriteLine($"Successfully transferred {amount} from Account {fromAccountId} to
{toAccountId}.");
}
catch (Exception ex)
{
    // ❌ Rollback on error
    transaction.Rollback();
    Console.WriteLine($"Transaction failed: {ex.Message}");
}
}

// Step 4: Display account balances
Console.WriteLine("\nAccount Balances:");
using (var cmd = new SQLiteCommand("SELECT AccountId, Name, Balance FROM Accounts", conn))
{
    using (var reader = cmd.ExecuteReader())
    {
        while (reader.Read())
        {
            Console.WriteLine($"Account {reader["AccountId"]} {reader["Name"]}: {reader["Balance"]}");
        }
    }
}

conn.Close();
}
}
}

```

7th CODE - SQL Injection Attack Simulation

```

using System;
using System.Data.SQLite;

```

```

class Program

```

```

{
    static string connectionString = "Data Source=security.db;Version=3;";

    static void Main()
    {
        SetupDatabase();

        Console.WriteLine("\n--- SQL Injection Attack ---");

        string attackInput = "' OR 1=1 --";

        VulnerableLogin(attackInput, "anything");
        SecureLogin(attackInput, "anything");
    }

    static void SetupDatabase()
    {
        using var conn = new SQLiteConnection(connectionString);
        conn.Open();

        string create = @"CREATE TABLE IF NOT EXISTS Users(
            Id INTEGER PRIMARY KEY,
            Username TEXT,
            Password TEXT
        );";

        new SQLiteCommand(create, conn).ExecuteNonQuery();

        string insert = @"INSERT OR IGNORE INTO Users(Id,Username,Password)
            VALUES(1,'admin','admin123');";

        new SQLiteCommand(insert, conn).ExecuteNonQuery();
    }
}

```

8th CODE - Thread-Safe Singleton Database Manager

```

using System;
using System.Data.SQLite;
using System.Threading;

class DatabaseManager
{

```

```

private static SQLiteConnection _instance = null;
private static readonly object _lock = new object();
private static bool _isCreated = false;

// Private constructor to prevent external instantiation
private DatabaseManager()
{
    Console.WriteLine($"[Thread {Thread.CurrentThread.ManagedThreadId}] DatabaseManager instance
created.");
    _instance = new SQLiteConnection("Data Source=singleton.db;Version=3;");
    _instance.Open();
}

// Public static method to get the singleton instance (thread-safe)
public static SQLiteConnection Instance
{
    get
    {
        if (_instance == null)
        {
            lock (_lock) // double-check locking
            {
                if (_instance == null)
                {
                    new DatabaseManager(); // initializes _instance
                }
            }
        }
        return _instance;
    }
}

class Program
{
    static void Main()
    {
        // Create multiple threads to access the singleton
        Thread t1 = new Thread(() => AccessDatabase());
        Thread t2 = new Thread(() => AccessDatabase());
        Thread t3 = new Thread(() => AccessDatabase());
    }
}

```

```

        t1.Start();
        t2.Start();
        t3.Start();

        t1.Join();
        t2.Join();
        t3.Join();
    }

    static void AccessDatabase()
    {
        var db = DatabaseManager.Instance;
        Console.WriteLine($"[Thread {Thread.CurrentThread.ManagedThreadId}] Using instance:
{db.GetHashCode()}");
    }
}

```

9th CODE - Extend Factory with Open-Closed Principle

```

public interface IShape
{
    void Draw();
}

using System;

public class Circle : IShape
{
    public void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}

public class Square : IShape
{
    public void Draw()
    {
        Console.WriteLine("Drawing Square");
    }
}

public class Triangle : IShape

```

```

{
    public void Draw()
    {
        Console.WriteLine("Drawing Triangle");
    }
}

using System;
using System.Collections.Generic;

public class ShapeFactory
{
    private readonly Dictionary<string, Func<IShape>> _shapeCreators =
        new Dictionary<string, Func<IShape>>();

    // Register shapes
    public void RegisterShape(string shapeName, Func<IShape> creator)
    {
        _shapeCreators[shapeName.ToLower()] = creator;
    }

    // Create shape
    public IShape CreateShape(string shapeName)
    {
        if (_shapeCreators.TryGetValue(shapeName.ToLower(), out var creator))
        {
            return creator();
        }

        throw new ArgumentException("Shape not registered");
    }
}

using System;

class Program
{
    static void Main()
    {
        ShapeFactory factory = new ShapeFactory();

        // Register shapes
        factory.RegisterShape("circle", () => new Circle());
        factory.RegisterShape("square", () => new Square());
    }
}

```

```
// Create shapes
IShape shape1 = factory.CreateShape("circle");
shape1.Draw();

IShape shape2 = factory.CreateShape("square");
shape2.Draw();

// Add new shape WITHOUT modifying factory
factory.RegisterShape("triangle", () => new Triangle());

IShape shape3 = factory.CreateShape("triangle");
shape3.Draw();
}
}
```

10th CODE - Observer-Based Stock Market

```
using System.Threading.Tasks;
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        Stock apple = new Stock("AAPL", 150);

        EmailSubscriber email = new EmailSubscriber();
        SmsSubscriber sms = new SmsSubscriber();

        apple.Subscribe(email);
        apple.Subscribe(sms);

        Console.WriteLine("Updating stock price...");

        apple.SetPrice(155);
        apple.SetPrice(160);

        Console.WriteLine("Main thread continues working...");

        Thread.Sleep(2000); // allow async tasks to finish
    }
}
```

}
}